

ORIE 7190 (AI for Science, Fall 2026) HW 1

Due: Friday 2026-09-11, 11:59pm, on Gradescope (course entry code 7BV55W)

Covers material through the deep learning lecture on Mon 2026-08-31. The vibe coding session on Wed 2026-09-02 will help with the code.

Hand in a python notebook, or a PDF with a code appendix.

Ask questions on Ed.

Recommended libraries: `numpy`, `pandas`, `matplotlib`, `scikit-learn`, `pytorch`

Questions (except #1) have two parts.

- **Part A — no AI assistance.** Answer these *before* you write the code for Part B. These will be graded on your reasoning, not on whether you turned out to be right.
- **Part B — AI encouraged.** Use whatever tools you like, as long as you can explain what your code is doing. **Note at the top which tools you used and where.**

Collaboration is fine. List who you worked with.

Question 1 — Dot Partition Associate

Dot Partition Associate is a game about fitting classifiers. You draw separating lines by hand, then work through the perceptron, the multilayer perceptron, and gradient-based fitting.

Play through assignment 3.11. Assignments 1 and 2 go with the supervised learning lecture; assignment 3 goes with deep learning. Hand in a screenshot showing you reached 3.11. The game keeps your progress in the browser, so play on one machine in one browser.

The game is a research prototype under active development. If something is broken or stuck, **post on Ed**.

Question 2 — How much data do I need?

Here the data is synthetic, so we know the right answer and can check what the fitting procedure recovers. Write a function that takes an integer `n` and:

- builds a dataframe of `n` rows with columns `x1`, `x2`, `y`;
- draws `x1` and `x2` uniformly on $[0, 1]$;
- draws `y` as a Bernoulli variable with $P(y = 1) = \text{logistic}(-5x_1 + 3x_2)$;
- fits $P(y = 1) = \text{logistic}(w_0 + w_1x_1 + w_2x_2)$ by logistic regression, without regularization;

- returns the fitted w_0, w_1, w_2 .

To turn regularization off, use `LogisticRegression(C=np.inf)`.

Part A (no AI)

Answer these **before running anything**.

1. You will call the function 1000 times with $n=100$ and histogram the three fitted coefficients. **Predict roughly where each histogram is centered**, and say why, in terms of how you generated the data.
2. **Predict how the variance of w_0 changes as n grows** from 20 to 200 — the shape of the curve, not just the direction. Why?
3. Now three targets. For each, say what you would change and what number you expect. You are choosing the settings; there is more than one right answer.
 - **(a) Make it overfit.** Choose a sample size n and a number of extra nonlinear features of x_1 and x_2 so that training accuracy beats held-out accuracy by **at least 20 points**. Both are yours to pick. Use `PolynomialFeatures(degree=7)` or higher — the lecture's random $\sin(a_j + b_j x)$ features are nearly collinear on $[0, 1]$ and only reach about 10 points. State the pair you would choose and the gap you expect, then predict what happens if you keep the features and multiply n by 10.
 - **(b) Make a correctly specified model perform badly anyway.** Keep the model exactly right, change only the data-generating distribution, and drive held-out accuracy toward 50%. What are you changing, and what sets the ceiling on accuracy that no model can beat?
 - **(c) Break your own evaluation.** Evaluate the model on the same rows you trained it on, at $n=50$, using the same features as in (a). Predict what accuracy this reports and what the honest number is. Which would you have reported if you were not paying attention?

Part B (AI encouraged)

4. Write the function. Call it 1000 times at $n=100$, plot the three histograms, report the means. Do the two slope means land exactly on the values you generated from? If not, in which direction are they off, and why would fitting on finite data do that?
5. For n on a grid from 20 to 200 — about ten values, not all 181 — call it 1000 times and plot the mean and variance of w_0 against n . With regularization off a small sample is often perfectly separable, and the coefficients then diverge rather than raising an error. Discard any replicate with $|w_0| > 10$ and **report how many you discarded at each n** . Expect a handful at $n = 20$ and essentially zero by $n = 40$. Say what that count measures.
6. Run your three experiments from (3) and report the numbers.

7. Where did your Part A predictions miss? One paragraph on what that tells you. This part carries the most weight.
-

Question 3 — How complex a model can your data afford?

The dataset

AqSolDB is a curated set of measured water solubilities for **9,982** compounds, mostly organic.

Sorkun, Khetan & Er. *AqSolDB, a curated reference set of aqueous solubility and 2D descriptors for a diverse set of compounds*. Scientific Data 6, 2019. doi:10.1038/s41597-019-0151-1 · data at <https://github.com/mcsorkun/AqSolDB>

Solubility measures how much of a solid can be dissolved into water before it stops dissolving. It is measured in moles per litre. A mole is a fixed number of molecules (6.022×10^{23}), so mol/L is how many molecules a litre of water can dissolve. The **Solubility** column is the base-10 logarithm of that number. The median compound here is -2.6 , about 0.0024 mol/L.

A **molecular descriptor** is a feature describing a molecule. We will return to these in the lecture on molecular representations and property prediction. Take these 17 as given.

Use this code to load the data:

```
import pandas as pd

URL = ("https://raw.githubusercontent.com/mcsorkun/AqSolDB/master/"
      "results/data_curated.csv")
df = pd.read_csv(URL)

DESCRIPTORS = ["MolWt", "MolLogP", "MolMR", "HeavyAtomCount", "NumHAcceptors",
               "NumHDonors", "NumHeteroatoms", "NumRotatableBonds",
               "NumValenceElectrons", "NumAromaticRings", "NumSaturatedRings",
               "NumAliphaticRings", "RingCount", "TPSA", "LabuteASA",
               "BalabanJ", "BertzCT"]

X = df[DESCRIPTORS].values
y = (df["Solubility"] > df["Solubility"].median()).astype(int).values
```

Several are quantities you might guess would matter for dissolving:

- **MolWt** — how heavy the molecule is.
- **MolLogP** — whether it would rather sit in oil or in water. Oily molecules dissolve poorly, so this one is close to the answer.

- **TPSA** — polar surface area: how much of the surface carries an uneven electrical charge. Water is itself lopsided in charge, so it sticks to polar surfaces.
- **NumHDonors, NumHAcceptors** — how many places can hydrogen-bond with water.
- **RingCount, NumRotatableBonds** and the other counts — size, shape, and flexibility.
- **BalabanJ, BertzCT** — how branched the pattern of connections between atoms is.

Two ways to make a linear model more flexible

The lecture covered both. In each, **model complexity** means the number of fitted parameters.

- **Logistic regression on polynomial features.** Add $x_i x_j$, x_i^2 , and so on up to degree d — `PolynomialFeatures(degree=d)`. With 17 descriptors, degree 1 is 18 parameters, degree 2 is 171, degree 3 is 1,140. Fit with `LogisticRegression(C=1e6, max_iter=20000)`; the default $C=1.0$ would regularize away the effect this question is about.
- **A multilayer perceptron.** Make it more flexible by widening a hidden layer or adding another one — `MLPClassifier(hidden_layer_sizes=...)`. For layer sizes $s_0, \dots, s_L$ the parameter count is $\sum_i (s_i s_{i+1} + s_{i+1})$.

Part A (no AI)

Answer these **before running anything**.

1. Training accuracy and held-out accuracy both estimate “how good is this model.” Which would you trust to say how the model does on a compound it has never seen, and why?
2. **Sketch, by hand, held-out error against number of parameters.** Draw one curve for 100 training compounds and one for 7,000. Mark roughly where the best model sits on each, and say which way that point moves as training data grows.
3. With 100 training compounds, the degree-3 model fits 1,140 parameters — more parameters than compounds. What do you expect its **training** accuracy to be? What does that say about choosing a model by training accuracy?
4. At 100 training compounds, do you expect the network or the polynomial model to do better? At 7,000? **Commit to both answers now.**

Part B (AI encouraged)

5. **Standardize the descriptors first.** The 17 columns are on very different scales — `MolWt` in the hundreds, `BertzCT` in the thousands, `NumHDonors` a small count. To *standardize* a column is to subtract its mean and divide

by its standard deviation, so each ends up centred at zero with a spread of about one.

```
from sklearn.preprocessing import StandardScaler
X = StandardScaler().fit_transform(X)
```

This is not cosmetic: unscaled, the degree-3 fit at $n = 7000$ runs for over twenty minutes without finishing and without raising an error.

6. Pick two of the standardized descriptors and make a scatter plot, one dot per compound, coloured by y . It is always worth plotting the data.
7. Hold out a fixed test set of 2,000 compounds and leave it alone for the whole question: `train_test_split(..., test_size=2000, stratify=y)`. This is the single-split version of the cross-validation from lecture. Then build both ladders — degrees 1 to 3, and hidden layers from (4,) to (256, 128) — recording the number of fitted parameters for each.
8. For each training-set size $n \in \{100, 300, 1000, 3000, 7000\}$, draw n compounds from what is left, fit every model, and score on the held-out test set. Which compounds you happen to draw changes the answer, so **repeat each fit on 5 independent draws and average them**, keeping the 5 individual scores — item 10 needs them. A few minutes on a laptop.
9. Report your results as two tables, one per family: rows are n , columns are the rungs of the ladder, entries are mean test accuracy.
10. **How big is the noise?** For each cell of those tables you have 5 scores. Report their standard deviation for a few representative cells. Then compare it against the spread from the best to the worst rung within a row. For which family, and at which n , are the differences between rungs larger than the noise?
11. **Plot test error against number of parameters — one curve per n , one panel per family**, with a log x-axis. Mark the lowest point on each curve. **Which way does the best complexity move as n grows?** Report the winning model at each n — and wherever item 10 showed the rungs are within noise of each other, say that instead of naming a winner.
12. Compare against your sketch and your answers in (3) and (4). Say where you were wrong.
13. Complete this sentence using your own numbers: “*You should use a more flexible model when _____.*”

Question 4 — What the network actually learned

The lecture claimed that a deep network is logistic regression applied to a representation the network learned for itself. If Question 3 gave you a network that beats logistic regression, this question asks where that advantage comes from.

Take the best MLP you found in Question 3 at the largest training-set size — if several were within noise of each other, any of them will do. Split it at the last layer. Everything before that layer is a function f turning 17 descriptors into a vector of hidden activations; the last layer is logistic regression on $f(x)$.

Fit four things on the same train/test split:

- (a) logistic regression on the 17 descriptors — the baseline.
- (b) the MLP, end to end.
- (c) logistic regression on the frozen representation $f(x)$ — push the training inputs through every layer but the last, freeze those weights, and fit a fresh logistic regression on the activations that come out.
- (d) the same thing on an **untrained** network — build a network of identical shape, do **not** fit it, and probe its randomly initialised representation the same way. This is the control.

Part A (no AI)

1. Predict all four accuracies, at least in order. Which is highest? Which two are closest?
2. From the architecture: can (c) ever come out much worse than (b)? What is the last layer of the network already doing?
3. Now the control. If (d) lands near (a), what does that tell you? If it lands near (c), what does that tell you, and would the lecture's claim survive it?

Part B (AI encouraged)

4. Run all four and report the accuracies.
5. In one or two sentences: what does the gap between (d) and (c) say about the lecture's claim? What would you have concluded from (a), (b) and (c) alone?

Use `MLPClassifier` or `PyTorch`, whichever you prefer. With `MLPClassifier` the learned weights are in `.coefs_` and `.intercepts_`, and the representation is what you get by pushing the input through every layer but the last, applying a ReLU after each — its default hidden activation.